WashKrew

Mac and iPhone continuation report

27 October 2026 | For Aman and Claude Code | Branch: staging

Application baseline: `81c40ee` (complete Flutter service connections and Mac handoff). This report and its PDF are a later documentation commit on the same branch. Use the latest `origin/staging`, not just the baseline commit. Repository: https://github.com/desicode-blip/wheelio-Both-Apps

Current position

The Customer and Captain software is ready for Mac compilation and physical-device validation. The web release and Firebase backend are published in test mode. An Xcode build, signed installation or physical iPhone pass has not been established on the Windows development machine. There are also external launch requirements; this is not a live-launch approval.

Latest owner decisions - read before older reports

Decision	What Claude must do
Staging design is locked	Preserve Customer/Captain composition, assets, typography, colors and components. Fix behavior in the existing screens. Do not restore the removed payment banner.
Aman will use training design placeholders for now	Obtain Aman's actual placeholder assets. Keep the existing training regions; identify missing content truthfully. A placeholder is not watched training or a passed quiz.
WashKrew needs a separate Razorpay gateway	The owner reports Razorpay said Trustkeyper's gateway cannot be shared. Aman is handling the new merchant application and documents. The old shared live setup is not an approved launch path.
Live payments remain blocked	Stay in test mode. New merchant access, integration, verification and explicit live acceptance are still required.
Flutter is the active stack	Work in `apps/flutter/`. The SwiftUI, Next.js and legacy Flutter projects are references/history. Android remains deferred.

Next work, in order

1. Pull the latest staging, read project instructions and prepare both Flutter iOS workspaces.
2. Resolve actual Xcode/CocoaPods/signing failures on the Mac without redesigning the apps.
3. Run the two-role iPhone checks and record reproducible failures before fixing them.
4. Add Aman's training placeholders without granting fake server training completion.
5. When Aman's separate merchant access is ready, implement and test the merchant migration described on page 6.
6. Close the remaining content, coverage, commercial and provider gates before any live launch.

1. What is already implemented

These are implemented service paths, not a claim that every native permission or provider combination has passed on a phone.

Area	Included behavior
Accounts	Firebase phone OTP, profiles/photos, same-account phone change, settings, sign-out and guarded Customer account deletion.
Bookings	Server catalog/prices/capacity, saved parking and pin coordinates, multiple requested windows, private notes/media and persisted orders.
Existing payment adapter	Server-created Razorpay orders, signature/capture checks, lost-callback recovery, original-method refunds, retries and webhook isolation. New merchant access is not connected.
Customer-Captain workflow	Applications/operator approval, expiring offers, atomic assignment, arrival, private before/after evidence, washing, Customer review/rating and earnings.
Tracking	Optional foreground GPS, real Google Routes ETA and stale/outage handling. Customer map dragging persists the selected pin. No continuous background tracking promise.
Photos and identity	Capture quality checks, private one-to-one reference comparison, uncertain-result human review and advisory vehicle-photo AI. No liveness certification or automatic KYC/dispute decision.
Training foundation	Real video playback, bounded server watch progress, versioned completion, server quiz grading and device speech answers. Approved clips are absent; placeholders are still to be supplied.
Support and operations	Private tickets/inbox, notification preferences/FCM code, disputes/held earnings, receipt/evidence downloads and reviewed operator records. Bank transfers remain manual operations.

Verification already completed

89 backend/rules tests, 24 Flutter tests (8 Customer, 14 shared Firebase, 1 Captain speech parser, 1 image-quality test), 3 actual local model tests, portable iOS tool checks, all app/shared/tool analyses and both release web builds passed on 27 October.

Authenticated local HTTP acceptance exercised assignment, a real local reference comparison, eight decoded evidence uploads, tracking/status, approval/rating/earnings and a single-use receipt. That fixture simulated payment capture and deliberately tested AI unavailability. Actual deployed private-model/Gemini checks and a public-location Routes request passed separately. Local browser checks verified restored accounts, the completed Captain job and the draggable/persisted Customer map.

Actual Razorpay test checkout/refunds were accepted on 21-22 October under the old merchant. That evidence does not validate the new WashKrew merchant or physical UPI switching. No production QA Captain, fixed OTP or real charge was fabricated. See `docs/WEB-ACCEPTANCE.md` for exact boundaries.

2. Mac preparation and compilation

Get the correct source

For a fresh checkout:

```
git clone https://github.com/desicode-blip/wheelio-Both-Apps.git
cd wheelio-Both-Apps
git switch staging
git pull --ff-only origin staging
```

For an existing checkout, inspect `git status` first and preserve local work. Fetch and fast-forward staging; do not use a hard reset to discard Mac changes. Record the actual commit from `git rev-parse HEAD` in the device test record.

Prepare, sign and build

Use Flutter 3.38.6 stable / Dart 3.10.7, Xcode with iOS platform support, and CocoaPods. Connect an unlocked trusted iPhone with Developer Mode enabled. The project deployment target is iOS 15+. Resolve relevant `flutter doctor -v` findings.

```
flutter doctor -v
bash apps/flutter/tool/run-ios.sh --prepare
open apps/flutter/customer/ios/Runner.xcworkspace
open apps/flutter/captain/ios/Runner.xcworkspace
```

Select the owning Apple team (saved ID `8CCQQM7TP3`) and correct provisioning in each workspace. Keep bundle IDs `com.amanmaheshwari.wheelio.customer` and `com.amanmaheshwari.wheelio.captain`. Do not build the sample SwiftUI `apps/ios/Wheelio.xcodeproj` for this acceptance.

The preparation script regenerates machine-local Flutter files, enforces Dart locks, refreshes pod specs and checks native plugins. The new Google Maps pods (both apps) and speech recognition pod (Captain) must be installed. Existing Mac pod locks are preserved in Git but have not been resolved for these additions. Fix a named incompatibility narrowly; do not broadly upgrade dependencies or delete all native project files. Commit successful Mac-resolved lock/project changes.

```
# Optional unsigned compilation, then signed Release install:
bash apps/flutter/tool/run-ios.sh --no-codesign
bash apps/flutter/tool/run-ios.sh
# With several phones, supply --device ID explicitly.
```

Enable Push Notifications and verify Remote notifications background mode for both targets. Upload the owning team's APNs key to the existing Firebase project; keep private signing material outside Git. Apply the documented geolocator Always-permission bypass before App Store submission. See `docs/MAC-HANDOFF.md` for the complete signing and APNs steps.

No Node, Firebase CLI login or Razorpay secret is required merely to compile these iPhone apps. Server deployment/content changes require separate owner access. Normal builds use Firebase; `--demo` is sample data and is not integration acceptance.

3. Physical iPhone acceptance still required

Prefer a Customer phone and a Captain phone; Customer web plus Captain iPhone is also useful. Use consenting accounts and an actually reviewed test-mode Captain. Do not seed fixed OTPs or invented approved Captains into the hosted project. If training/content blocks an application, exercise it in isolated QA rather than silently bypassing production approval.

Check	Required observation
Cold Release launch	Both newly built apps open after disconnecting Xcode. Branding/layout match staging; no accidental demo data.
SMS and account recovery	Real OTP, resend, wrong/expired code, edit number, profile save, sign-out/reopen and same-account phone update behave correctly.
Camera/photos	Allow, deny and re-enable permissions; cancel capture; replace Customer, KYC, selfie and all before/after photos. A cancelled/failed capture never counts as uploaded.
Voice/video/speech	Record, play, remove and retry private media. Test quiz speech in English/Hindi/Telugu and denied/unavailable recognition; tap answers remain usable.
Maps/location	Native tiles/pin render, confirmed pin restores, permissions time out safely, foreground GPS reaches Customer and stale location is clear after backgrounding.
Checkout	Correct server amount/order, chosen instrument, cancellation/retry, UPI app switch/return, network loss and independent payment-status recovery. Use authorized test mode only.
Assignment/privacy	One Captain claims the offer; both roles show the same order. Other users cannot read private parking, media or tracking. Cancellation removes stale wash controls.
Identity/evidence	Consenting Captain reference match, poor/different capture and manual-review paths. Four before photos gate start; four after photos gate finish. Correct-side retakes persist.
Review and money	Photo warnings/outage acknowledgement, Customer before/after review, approve/rate once, earnings and a dispute hold. No fake bank transfer or model approval.
Refund/retry	A refundable cancellation reaches the actual test-provider result; repeated status checks/retries do not duplicate refunds or bookings. Repeat on the new merchant after integration.
Push/inbox	APNs foreground/background delivery, taps, denied permission, preferences, token/sign-out behavior and persisted read state.
Reopen/time boundaries	Restore an active job, reject future-slot arrival/start, test expired offers and Customer cancellation during a Captain screen.

Record commit, Xcode/Flutter/iOS versions, device models, both roles, steps, expected/actual results and redacted evidence. Mark each case PASS, FAIL or BLOCKED. A blocked case is not a pass. Never include OTPs, keys, personal KYC images or private phone details in the shared report.

4. Training placeholders and other pending inputs

Aman's temporary training design placeholders

The owner says Aman will provide design placeholders instead of videos for now. No placeholder assets or replacement training content have been supplied in this handoff. Claude should use Aman's supplied assets in the existing W09 training/video region, retaining the staging layout and tokens. Do not invent a new design, sample lesson or working video URL.

Keep missing/playback-unavailable states honest. A static placeholder must not emit watch-progress heartbeats, write `trainingCompletedAt` / `quizPassedAt`, simulate a successful stream or auto-approve a Captain. Do not restore the six-second demo timer in Firebase mode. If a separate isolated QA preview is needed, label and document it; do not silently relax production training/KYC gates. A change to the real training requirement needs an explicit owner decision and documented server policy.

The implemented video/quiz path should remain available for real clips later. Current content publication expects three direct HTTPS media URLs and real durations, plus the approved version. A placeholder image is not valid video content for that publisher. Review quiz questions against the eventual approved lessons.

What still needs decisions or content

Owner/input	Remaining item
Aman	Placeholder assets now; approved training approach/content later.
Aman	Operating legal entity, support phone/email/hours, address and approved terms/privacy/cancellation/refund policies.
Owner/Aman	Live customer prices, Captain payouts, cancellation/trip-fee/penalty/level rules and verified introduction-call availability. Existing test values are not approved live terms.
Owner/operations	Actual served societies, accurate boundaries/coordinates and approved real Captains. My Home Avatar's map location is verified; its viewport is not a service geofence. Avali/Avishkaar matches still need review.
Apple team/Aman	Signing access, APNs configuration and phone acceptance.
Provider accounts, deferred by owner	DigiLocker, WhatsApp Business, voice-call OTP and masked Customer-Captain calls. Do not claim these are connected.

Support/policy configuration is wired but blank values must not become dummy numbers or invented promises. Use `docs/APPROVED-CONTENT.example.json` only after the required values are approved. The owner-only `publish-content` command previews before `--confirm`. Payout transfers, KYC/work eligibility and dispute outcomes remain reviewed human operations.

Separate refund-to-a-new-UPI-ID is not connected; current cash refunds return to the original payment method. Any alternative needs approved policy/provider support and reconciliation. Pre-acceptance offers intentionally hide exact plate/parking; keep that privacy boundary. DigiLocker remains deferred; private photographed documents are the current application path.

Photo-library controls or an additional live-location screen need the owner's design decision if they require new UI. Optional foreground sharing exists; continuous background GPS was not implemented or promised. Android generation and App Store distribution/submission are outside this Mac test handoff.

5. Separate Razorpay gateway - pending migration

New direction and current state

The owner reports Razorpay advised that WashKrew cannot use Trustkeyper's payment gateway. Aman is now handling the separate WashKrew merchant application, supporting documents and access generation. Do not restart or duplicate his onboarding. No new merchant ID, approval or credentials have been supplied or connected in this source.

Earlier notes about the shared live pair and website review describe historical setup. They are not authorization to launch WashKrew on that merchant. The current server remains test mode; its old archived test credentials still support existing test records. No payment credential or secret is in this report.

Handoff from Aman when the new account is ready

Obtain the new merchant ID, approved operating identity/website, access for the responsible developer, separate test/live API credentials and a dedicated webhook signing secret through secure account/Secret Manager access. Aman completes merchant KYC/OTP/bank verification directly. Do not paste keys, PAN/Aadhaar or bank documents into Git, PDFs or chat. Keep the existing Firebase project/billing unless the owner asks otherwise.

Work Claude must do before switching

1. Verify the new credentials belong to the new merchant and expected test/live mode without charging funds. Store secrets server-side; no secret belongs in Flutter, web defines or committed JSON.
2. Audit outstanding old checkouts, captures, queued refunds and webhook retries. The current `getPayments(mode)` / `paymentProviders` wiring distinguishes test from live, not two merchants in the same mode. This is a real migration requirement, not already implemented merchant-version routing.
3. Implement and test a reviewed merchant/version mapping on new orders and refund processing, or another explicitly reviewed cutover that preserves every old obligation. Historical orders must retain their original provider and idempotency references. Merely replacing the archived test secret breaks old reconciliation/refunds. Do not relabel old orders as new-merchant or live orders.
4. Register new-merchant webhook subscriptions with their own signing verification for `payment.captured`, `order.paid` and `refund.processed`. Plan coexistence/retries and preserve Trustkeyper's settings. Verify raw-body signatures, amount/currency/order ownership, mode and merchant isolation. Do not retire old endpoints before outstanding work is reconciled.
5. Use the new merchant's test mode for fresh Customer checkout, callback loss, failed/cancelled checkout, capture, assignment, full refund/retry and duplicate-event acceptance. Check both web and native UPI/card behavior. The old merchant's acceptance cannot substitute for this.
6. Review deployment bindings/configuration and the domain-policy-compatible gateway step. Keep payment mode test until merchant approval, actual commercial/content/coverage decisions, approved live Captains and explicitly authorized live checkout/refund acceptance are complete.

Read `backend/firebase/functions/src/index.mjs`, `service.mjs`, `payments.mjs`, payment tests and the current launch report before changing secrets. Do not regenerate Trustkeyper's keys, reuse a masked secret or assume Razorpay Route/bank payouts are automatically configured. No new merchant integration or live transaction was performed by this handoff task.

6. Claude Code starting instructions

Paste this into Claude Code after pulling the latest staging on Aman's Mac:

```
Read AGENTS.md, CLAUDE.md and
docs/CLAUDE-MAC-CONTINUATION-2026-10-27.md first.
Use apps/flutter Customer and Captain, preserving the staging
design exactly. Read docs/MAC-HANDOFF.md and prepare/build both
iOS workspaces on this Mac. Fix actual compilation and device
failures; keep a clear PASS/FAIL/BLOCKED acceptance record.
Use Aman's supplied training placeholders only in the existing
training regions. Never fake video completion, quiz success,
KYC approval, payments, location or bank transfers.
Aman is handling the separate WashKrew Razorpay account.
Keep test mode. Do not activate the old Trustkeyper merchant.
When new access is provided, implement merchant-safe payment
and refund routing and repeat new-merchant acceptance first.
Keep secrets/signing keys outside Git. Do not generate Android.
Commit changes and update the relevant project status docs.
```

Where to continue

Work	Primary source/reference
Mac build / native configuration	`docs/MAC-HANDOFF.md`; `apps/flutter/tool/ios.dart`; Customer/Captain `ios/Runner` and workspaces.
Designed screens	Customer/Captain `lib/screens`; shared `packages/washkrew_ui`; staging assets/tokens.
Firebase / native Auth / checkout	`apps/flutter/packages/washkrew_firebase/lib/src`; native options and checkout adapters.
Training / quiz / speech	Captain `lib/screens/onboarding.dart`, `lib/data/spoken_answer.dart`; backend `training.mjs`, `content.mjs`.
Maps / photos	Shared `live_map.dart`; backend `maps.mjs`, `identity.mjs`, `wash-photo-review.mjs`; `backend/photo-check/README.md`.
Payment migration / operator work	Backend `index.mjs`, `service.mjs`, `payments.mjs`, `operator.cjs`, `operations.cjs` and payment tests.
Evidence and old checklists	`docs/WEB-ACCEPTANCE.md`, `READINESS-2026-10-27.md`, `BACKEND-HOOKUP.md`, `P0-BACKEND-FOR-DESIGN.md`. Older unticked rows are not proof that current server paths are absent.

Closeout requirements

Run checks appropriate to changed code. Existing commands include `flutter analyze`, each package's `flutter test`, `dart apps/flutter/tool/ios_test.dart` and the Firebase emulator suite documented in `docs/FIREBASE-BOOKINGS.md` (Java 21+). Never point QA fixtures at production. Record new failures and fixes separately from the previous passing evidence.

Commit every source change; update `docs/FLUTTER.md` and `docs/BACKEND-HOOKUP.md` for app changes and explicit placeholders/fakes. Record Mac-resolved pod locks after compilation. Keep the actual tested commit and device evidence in the acceptance report. Distinguish software completion, physical-device acceptance and live-business approval. This report contains no signing keys, payment secrets or private KYC documents.